

Generating Test Sequences from UML Use Case Diagram: A Case Study

Zahra Abdulkarim Hamza
Department of Computer Science
University of Bahrain
Sakhir, Kingdom of Bahrain
20112417@stu.uob.edu.bh

Mustafa Hammad
Department of Computer Science
University of Bahrain
Sakhir, Kingdom of Bahrain
mhammad@uob.edu.bh

Abstract—Testing is one of the main topics in software engineering. To perform testing tasks, a set of pre-testing steps should be done as testing preparation. One of the pre-testing steps is test sequence generation. In this paper, a case study has been used to apply a previously proposed approach of test sequence generation. The approach is based on the UML use case model analysis. The generation process includes UML use case diagram analysis, a conversion process into activity diagrams, simplifying the activity diagrams, and an information extraction process.

Keywords—Software testing, UML use case, test sequence, software engineering

I. INTRODUCTION

Testing software artifacts is one of the Software Development Life-Cycle (SDLC). Some tasks are required to be done as pre-testing steps, such as generating test sequences.

Software testing is a mandatory phase to make sure of the software conformance to its requirements, nevertheless, it may cause more issues if it is done in a wrong way. Generating test sequences helps in identifying the testing paths of the under-implementation software. Software testers need the test sequences to follow them while testing the software. Although test sequences can lead to effective testing, it is costly in terms of money and efforts, and it is time-consuming. More than an approach has been proposed to resolve the problem but many of them without detailed case studies.

In this paper, a case study will be discussed by applying an approach presented in [1] and [2]. The chosen case study is Online-Shopping [3]. It is a well-known case study and it has been used in explaining many approaches related to software engineering. Using the selected case study, the approach will be explained in details by going through each phase and step. In the end, the results will be shown.

This research paper is divided into several sections. The next section is a review of the field literature. After that, the approach is explained in a section. The following section consists of subsections that explains the case study. In the end, the conclusion section comes to wrap the research paper.

II. LITERATURE REVIEW

Many approaches have been proposed to generate test sequences. The input of the test sequences generation process varies between an approach to another. Most of the literature focuses on the test case generation, but it is helpful to review how the proposed approaches are dealing with the UML models. This section presents a literature review of some of the previously proposed approaches.

Some approaches use the behavioral UML models to in testing phase. Swain and Mohapatra [4] proposed an approach based on UML sequence and activity diagrams. The process starts with converting the behavioral models into graphs. Then, test scenarios are generated using an algorithm. After that, the test sequences are generated. This approach reduces the number of test cases by keeping the test coverage as it is. Another work [5] proposed an approach that is having the same idea of converting the UML sequence and activity diagram into graphs to generate test cases. However, the second phase is to integrate graphs to fully extract the needed information. In the end, Breadth-First Search is used to extract the test paths. Moreover, a survey [6] has been conducted to review the behavioral models used in the test case generation and other test-related data extraction. The survey considered the UML state, sequences, and activity models. Such a survey is conducted to help in understanding the reusability of the UML behavioral models in the testing phase of SDLC. UML state diagrams is also a behavioral model that is used in [7] to generate test cases. In such an approach, the control and flow information of the state diagram is used to find out the information of the test cases. Some test case generation approaches are proposed based on genetic algorithms along with the UML state diagrams, such as the work in [8]. Furthermore, an approach proposed in [9] depends on the depth-first search algorithm along with UML activity and sequence diagrams. The results are promising and software testers can depend on them. Another work that used the UML sequence diagram is shown in [10]. The proposed approach is automatically generated test cases from the UML diagram.

UML different models can also be used together to get better results. One of the proposed approaches [11] used different UML models to generate test cases. Such an approach provides a proof to generate test cases from the software design. Furthermore, an approach [12] has been proposed depending on the integration of UML class and sequence diagrams. The test cases are generated through combining the variable assignment graph. Another work [13] proposed an approach with the same idea of UML models' combinations. The approach shows that using UML use case and sequence diagrams, the test cases can be generated after converting them to graphs. Moreover, Khurana et al. [14] proposed a novel approach depends on several UML diagrams, which are use case, activity, and sequence. After generating separated graphs from each of the diagrams, the information is extracted to generate the test cases. Besides, Jagtap et al. [15] studied generating test cases from UML use case and state diagrams. The main feature of this approach, which is the same as the other approaches, that test cases can be generated before even the coding part.

UML behavioral models have been used along with UML functional models for testing preparations in some approaches. Dhineshkumar [16] proposed an approach to generate test cases using a UML sequence diagram. In this approach, relational definition language and UML use case diagram, which is a functional model, is used in parallel with the UML sequence diagram. The sequence diagram is converted to a graph to generate, at the end, the test cases. Another approach [17] proposed is based on UML use case specifications. The approach automatically produces executable test cases. Moreover,

UML functional models are also used as a basis for testing purposes. The work in [18] presented an approach for test case generation by analyzing UML use case diagrams. The approach has been tested with a project. The results of applying the approach to the selected project shows that most of the generated test cases are in the actual test cases' list. A new approach that also uses the UML use case is proposed in [19]. In this approach, the UML use case specifications are analyzed and converted into activity diagrams for each actor to generate test sequences.

As generating the UML use case diagrams is the pre-step of using it in test sequences generation, there are many proposed approaches for such purpose. For example, the work in [20] proposed an approach to generate UML use case by analyzing software requirements using natural language processing. The selected approach for generating the test sequences [1] [2] can be considered as the following process for the UML use case generation. There is more testing related work, such as the reviewed testing techniques in [21] and [22].

III. TEST SEQUENCES GENERATION

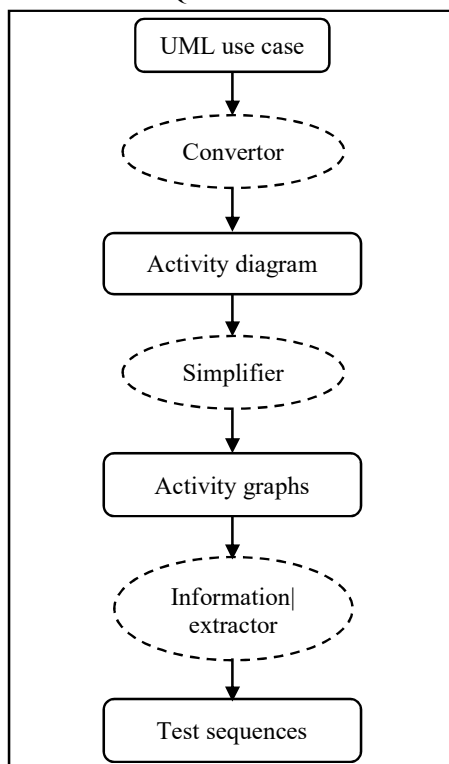


Fig. 1. Selected approach for generating test sequences

The selected approach, which is applied to the case study, is proposed in [1] and extended in [2]. Figure 1 illustrates the

included processes of the approach. As shown, UML use case is the input for the test sequences' generation. For each actor, an activity diagram is created. Then, the activity diagrams are simplified to activity graphs. Each activity graph is representing exactly one test path. In the end, the test sequences are generated using the extracted information. In the next section, each process will be explained in steps by applying the approach to the Online Shopping.

IV. CASE STUDY

Online-Shopping [3] is a well-known case study that is used in the software engineering research field. Such a case study has many different scenarios that are common between other case studies. The scenarios' mutuality feature makes the Online-Shopping an appropriate choice for evaluating the test sequences generation approaches.

TABLE I. ONLINE-SHOPPING UML USE CASE INFORMATION

Project	Use-cases	Actors	Actor – Use-case relationships	Use-case – use-case relationship
Online shopping	20	5	9	11

Online-Shopping system has multiple users and different functionalities. From the UML functional models angle, Online-Shopping UML use case diagrams consist of different actors and use cases. There are twenty use cases, five actors, nine actor-use-case relationships, and eleven relationships between the use-cases themselves, as shown in Table 1.

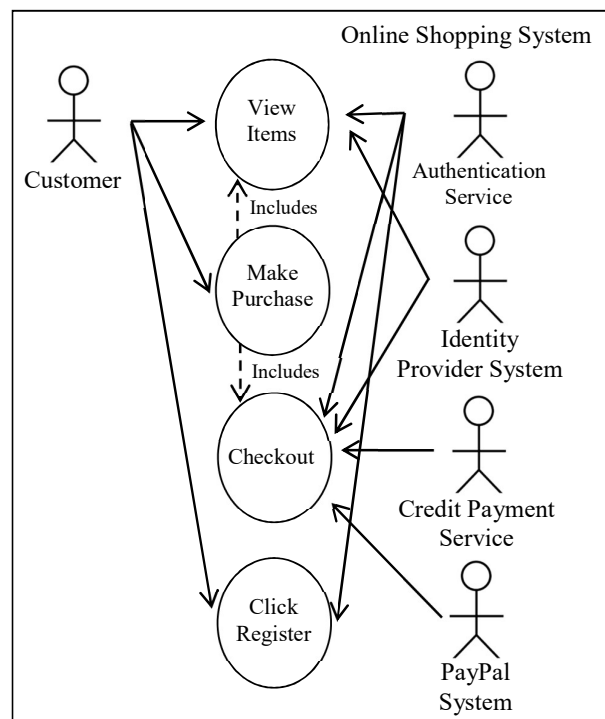


Fig. 2. UML use case diagram for Online-Shopping

The UML use case diagram of the Online-Shopping [3] system is shown in Figure 2. The use-cases are {View items, Make purchase, Checkout, Client register}. "View items" and

“Checkout” use-cases are a set of use cases, as shown in Figure 3 and Figure 4. However, the actors are {Customer, Authentication “service”, identity provider “system”, Credit payment “service”, PayPal “system”}.

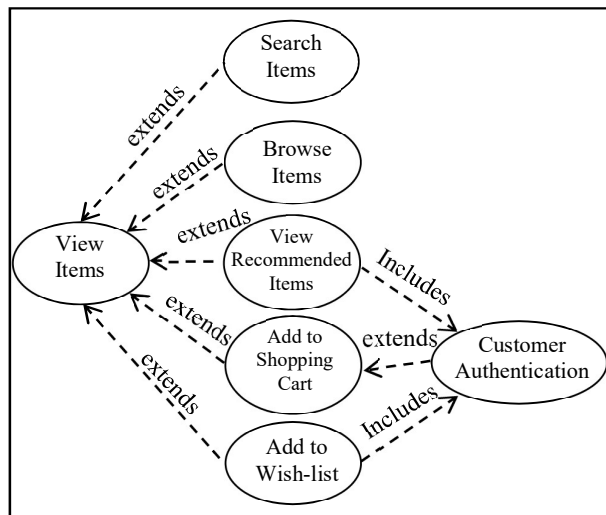


Fig. 3. All related use cases to “View items” use case

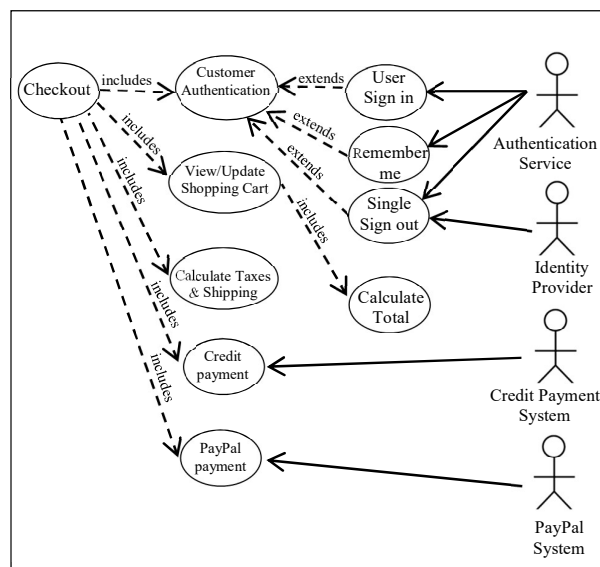


Fig. 4. All related use cases to “View items” use case

A. Conversion process

In the conversion process, for each actor in the UML use case diagram, a customized activity diagram is generated. Five activity diagrams will be generated as there are five actors in the UML use case diagram of the Online-Shopping system.

The first activity diagram is generated for the “Customer” actor, as shown in Figure 5. In Figure 2, the actor “Customer” is interacting with the system through three use cases directly. One of the use case “View items” consists of several use cases, which are shown in Figure 3. Therefore, there are many nodes related to “View items” due to the “extends” relationship with other use cases. The same case can be seen obviously with the “Make purchase” use case, which “includes” the use case

“Checkout”. Besides, the “Checkout” use case has many use cases that extend it.

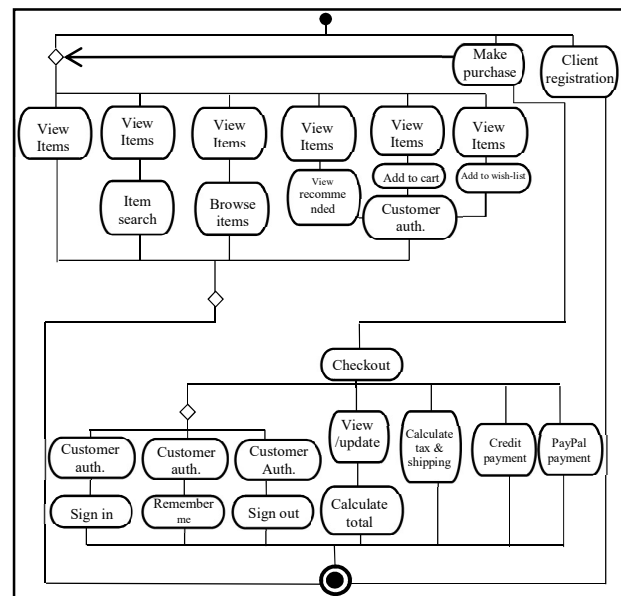


Fig. 5. Generated activity diagram for “Customer”

The second activity diagram is for the “Authentication service” actor, as shown in Figure 6. In this activity diagram, the actor has relationships with three use cases {View items, Checkout, Client register}. As mentioned previously, “View items” and “Checkout” use-cases have relationships with other use cases through “extends” and “includes” relationships. However, in this activity diagram, the relationships are with other use cases than the ones of the “Customer” activity diagram, due to the system functionality and actor purposes.

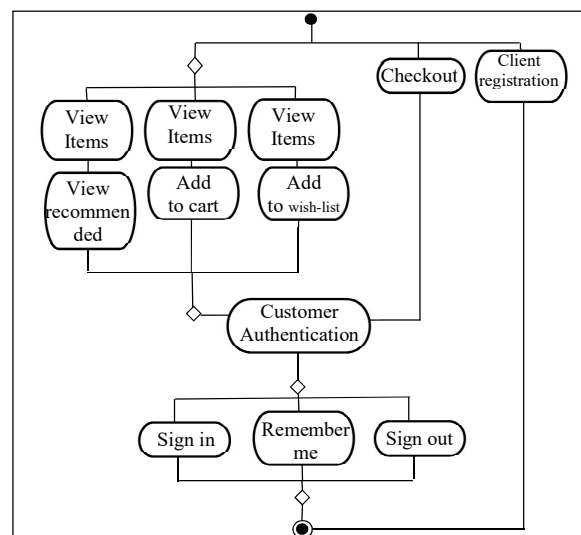


Fig. 6. Generated activity diagram for “Authentication service”

The third activity diagram is for the “Identity provider” actor, as shown in Figure 7. The “Identity provider” is connected to “View items” and “Checkout” use cases. Each of

the use cases are connected to other use cases through “extends” and “includes”.

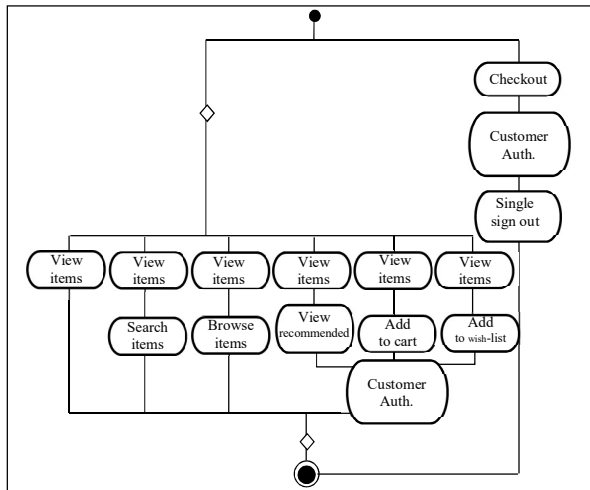


Fig. 7. Generated Activity diagram for the actor “Identity provider”

The last two activity diagrams are for the payment use cases. One is for the “Credit payment” actor. Such an activity diagram, as shown in Figure 8 has only one connection to a use case that “includes” another one, which is “Checkout” and “Credit payment” respectively. The other activity diagram is for the “PayPal payment” actor, which has the same functions as the “Credit payment” actor. As shown in Figure 9, the actor is connected to the “Checkout” that “includes” the use case “PayPal payment”.

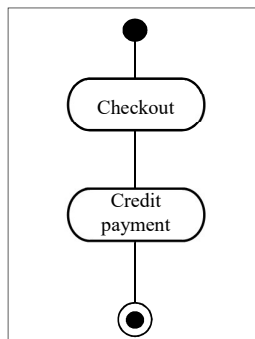


Fig. 8. Generated Activity diagram for actor “Credit payment service”

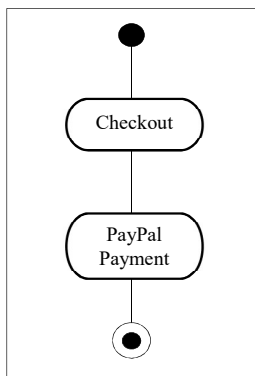


Fig. 9. Generated Activity diagram for the actor “PayPal payment service”

B. Simplification process

To simplify the problem of information extraction, later on, each of the previously generated activity diagrams are converted into activity graphs. For each actor, there is a set of activity graphs that represents the test paths/ sequences.

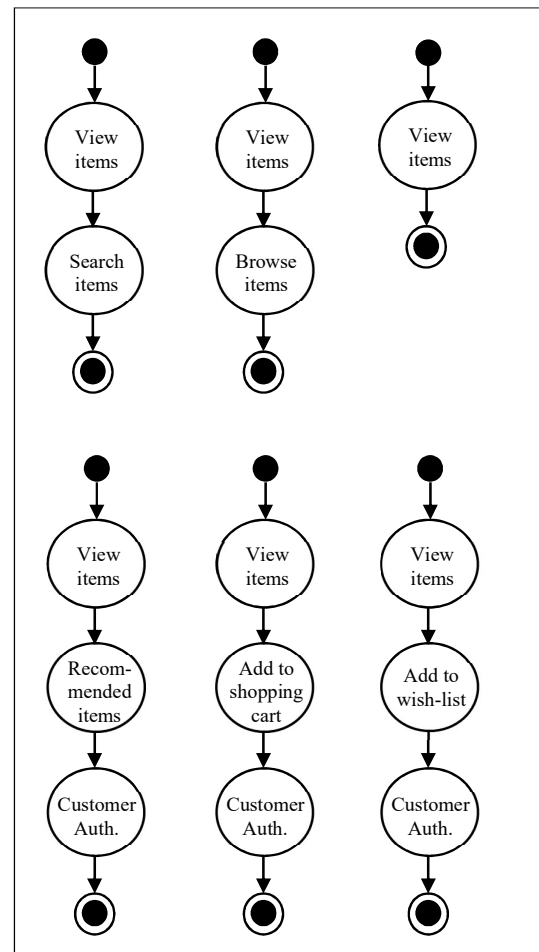


Fig. 10. Part of the generated activity graphs

Part of the generated activity graphs are shown in Figure 10. They are for the actor “Customer”. All of the “Customer” and other actors’ test sequences are shown in sub-section C, Table 2. As shown in Figure 10, the activity graphs are directed graphs and represent only on the test path, which is one test sequence.

C. Information extraction process

The last process after simplifying the activity diagram and generating the activity graphs is the information extraction. Each of the activity graphs is representing exactly one test path, which is a test sequence.

Table 2 represents the test sequences that are generated from the activity graphs shown in Figure 10. Nineteen test sequences have been generated for the actor “Customer”. Six of the test sequences start with “View items” and thirteen of them are related to “Make purchase”. However, there is only one test sequence for the client registration.

TABLE II. GENERATED TEST SEQUENCES FOR “CUSTOMER”

TS #	Actor	Test Sequence
TS1	Customer	View items → END
TS2		View items → Search items → END
TS3		View items → Browse items → END
TS4		View items → View recommended items → Customer authentication → END
TS5		View items → Add to shopping cart → Customer authentication → END
TS6		View items → Add to wish-list → Customer authentication → END
TS7		Make purchase → View items → END
TS8		Make purchase → View items → Search items → END
TS9		Make purchase → View items → Browse items → END
TS10		Make purchase → View items → View recommended items → Customer authentication → END
TS11		Make purchase → View items → Add to shopping cart → Customer authentication → END
TS12		Make purchase → View items → Add to wish-list → Customer authentication → END
TS13		Make purchase → Checkout → Customer Authentication → Sign-in → END
TS14		Make purchase → Checkout → Customer Authentication → Remember me → END
TS15		Make purchase → Checkout → Customer Authentication → Single Sign-out → END
TS16		Make purchase → Checkout → View/update shopping cart → Calculate total → END
TS17		Make purchase → Checkout → Calculate taxes and shipping → END
TS18		Make purchase → Checkout → Credit payment → END
TS19		Make purchase → Checkout → PayPal payment → END
TS20		Client register → END

Seven test sequences have been generated for the actor “Authentication service”, as represented in Table 3. Six of them are related to the “View items” use case and one for the client registration. All of the test sequence of “Authentication service” can also be seen in the “Customer” test sequences. The redundancy of the test sequences for more than an actor does not mean they are the same. All the test sequences should be followed while testing from each actors’ perspective.

TABLE III. GENERATED TEST SEQUENCE FOR “AUTHENTICATION SERVICE”

TS #	Actor	Test Sequence
TS1	Authentication Service	View items → View recommended items → Customer authentication → END
TS2		View items → Add to shopping cart → Customer authentication → END
TS3		View items → Add to wish-list → Customer authentication → END
TS4		Checkout → Customer Authentication → Sign-in → END
TS5		Checkout → Customer Authentication → Remember me → END
TS6		Checkout → Customer Authentication → Single Sign-out → END
TS7		Client register → END

“Identity provider” is the third actor in the Online-Shopping system. There are six generated test sequences for this actor. It is clear that the test sequences are also part of other actors’ test sequences, but all of them should be tested with consideration of the actor. However, only one test sequence is different, which is related to “Single sign-out”.

TABLE IV. GENERATED TEST SEQUENCE FOR “IDENTITY PROVIDER”

TS #	Actor	Test Sequence
TS1	Identity provider	View items → END
TS2		View items → Search items → END
TS3		View items → Browse items → END
TS4		View items → View recommended items → Customer authentication → END
TS5		View items → Add to shopping cart → Customer authentication → END
TS6		View items → Add to wish-list → Customer authentication → END
TS7		Checkout → Customer Authentication → Single Sign-out → END

Table 5 and Table 6 show the test sequences related to the payment use cases. The same test sequence is for both actors “Credit payment service” and “PayPal payment service”. However, the test sequences look like each other, but actually, they are different due to the difference of actors.

TABLE V. GENERATED TEST SEQUENCE FOR "CREDIT PAYMENT"

TS #	Actor	Test Sequence
TS1	Credit payment service	Checkout → Credit payment → END

TABLE VI. GENERATED TEST SEQUENCE FOR "PAYPAL PAYMENT"

TS #	Actor	Test Sequence
TS1	PayPal payment service	Checkout → PayPal payment → END

V. CONCLUSION AND FUTURE WORK

In this paper, Online-Shopping has been used as a case study to apply an approach of generating test sequences. The approach was proposed previously in [1]. The selected case study (Online-Shopping) has many common scenarios that already exist in other case studies, which makes it an appropriate case study for such an approach. This paper presents a helpful case study for software engineers and especially the software testers.

Some obstacles limit the work in this paper. The generated test sequences can differ if the order of the use cases occurrences is considered. Moreover, software engineers can generate different diagrams from the same software requirements, which we did not consider in this case study.

Some improvements can be done to this case study. First of all, we can include more scenarios from the other Online-Shopping projects that have some differences to cover more functionalities. Furthermore, the order of the occurrences of the use cases should be considered to see if it affects the test sequences.

REFERENCES

- [1] Z. Hamza and M. Hammad, "Generating test sequences from UML use-case diagrams," in 2019 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT) - IEEE, Sakhr, 2019.
- [2] Z. Hamza and M. Hammad, "Analyzing UML use cases to generate test sequences," International Journal of Computing and Digital Systems, 2020.
- [3] UML-diagrams-examples, "UML diagrams," 15 4 2020. [Online]. Available: <https://www.uml-diagrams.org/use-case-diagrams-examples.html>.
- [4] S. Swain and D. Mohapatra, "Test case generation from Behavioral UML Models," International Journal of computer applications, vol. 6, no. 8, pp. 5-11, 2010.
- [5] V. Sumalatha and G. Raju, "Uml based automated test case generation technique using activity-sequence diagram," The International Journal of Computer Science & Applications (TIJCSA), vol. 1, no. 9, 2012.
- [6] M. Shirole and R. Kumar, "UML behavioral model based test case generation: a survey," ACM SIGSOFT Software Engineering Notes, vol. 38, no. 4, pp. 1-13, 2013.
- [7] YG-Kim, HS-Hong, DH-Bae and SD-Cha, "Test cases generation from UML state diagrams," in IEE Proceedings-Software, 1999.
- [8] M. Shirole, A. Suthar and R. Kumar, "Generation of improved test cases from UML state diagram using genetic algorithm," in In Proceedings of the 4th India Software Engineering Conference, 2011.
- [9] I. Septian, R. Alianto and F. Gaol, "Automated test case generation from UML activity diagram and sequence diagram using depth first search algorithm," in Procedia computer science, 2017.
- [10] V. Panthi and D. Mohapatra, "Automatic test case generation using sequence diagram," in International Conference on Advances in Computing - Springer, New Delhi, 2013.
- [11] Abdurazik and Offutt, "Using UML Collaboration Diagrams for Static Checking and Test Generation," in in International conference on the unified modeling language, Berlin, 2000.
- [12] G. Sudipto, R. France, C. Braganza, N. Kawane, A. Andrews and O. Pilskalns, "Test adequacy assessment for UML design model testing," in in 14th International Symposium on Software Reliability Engineering, 2003.
- [13] Swain, Mohapatra and Mall, "Test Case Generation Based on Use case and sequence diagrams," International Journal of Software Engineering 3(2), pp. 21-52, 2010.
- [14] N. Khurana, R. Chhillar and U. Chhillar, "A Novel Technique for Generation and Optimization of Test Cases Using Use Case, Sequence, Activity Diagram and Genetic Algorithm," JWS, vol. 11, no. 3, pp. 242-250, 2016.
- [15] S. Jagtap, V. Gawade, R. Pawar, S. Shendge and P. Avhad, "Generate Test Cases From UML Use Case and State Chart Diagrams," International Research Journal of Engineering and Technology (IRJET), vol. 3, no. 10, pp. 873-881, 2016.
- [16] M. Dhineshkumar, "An approach to generate test cases from sequence diagram," in 2014 International Conference on Intelligent Computing Applications - IEEE, 2014.
- [17] C. Wang, F. Pastore, A. Goknil, L. Briand and Z. Iqbal, "Automatic generation of system test cases from use case specifications," in International Symposium on Software Testing and Analysis, 2015.
- [18] Hasling, Goetz and Beetz, "Model based testing of system requirements using UML use case models," in in 1st International Conference on Software Testing, Verification, and Validation, 2008.
- [19] Tiwari and Gupta, "An Approach of Generating Test Requirements for Agile Software Development," in in Proceedings of the 8th India Software Engineering Conference, 2015.
- [20] Z. Abdulkarim Hamza and M. Hammad, "Generating UML Use Case Models from Software Requirements Using Natural Language Processing," in in Proceedings of International Conference on Modeling Simulation and Applied Optimization (ICMSAO), 2019.
- [21] Z. Hamza and M. Hammad, "Web and Mobile Applications' Testing using Black and White Box approaches," in Smart cities 2019, 2019.
- [22] Z. Hamza and M. Hammad, "Testing Approaches for Web and Mobile Applications: An Overview," International Journal of Computing and Digital Systems, vol. 9, no. 4, pp. 657-664, 2020.